BIRZEIT UNIVERSITY

Faculty of Engineering and Technology

Electrical and Computer Engineering Department

Digital Signal Processing (DSP) - ENCS4310

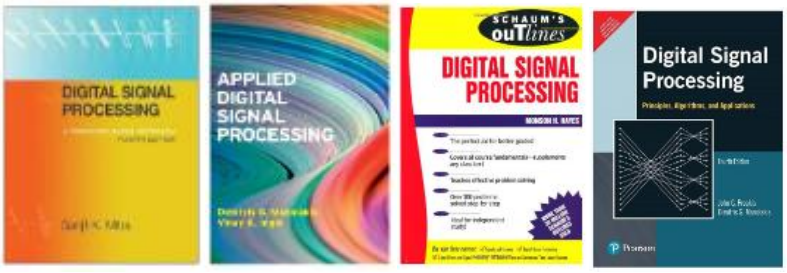
<u>Assignment 1</u>

Name: Nagham Moutaz Maali

ID: 1212312

Section: 2

Instructor: Dr. Qadri Mayyala



Contents

1. Introduction 2
2. Cross-Correlation and Autocorrelation 2
3. Task 1: Correlation vs. Convolution 5
   3.1. Mathematical relation 5
   3.2. Key difference 6
   3.3. Typical applications 7
4. Similarity Detection Problem 8
5. Normalized Correlation 9
6. Task 2: MATLAB Analysis 11
7. Conclusion 18

1. Introduction

Digital Signal Processing (DSP) is an important field in electrical and computer engineering that focuses on analyzing and processing signals such as audio, images, and communication data. One of the key tasks in DSP is to detect and compare signals, especially when they are affected by noise or distortion.

A common way to measure similarity between signals is through correlation. Cross-correlation is used to determine how similar two signals are, especially when one signal may be shifted in time (delayed). Autocorrelation is a special case where a signal is compared with itself, and it is useful for detecting patterns such as periodicity.

These concepts are widely used in real-world applications such as radar systems, where a transmitted signal is reflected back from an object and received with noise and delay. By using correlation, the system can detect the presence of the original signal and determine its position in time, as illustrated in the figure.

In this assignment, we study the mathematical definitions of cross-correlation and autocorrelation, and explore their relationship with convolution. We also examine how correlation can be used to measure similarity between signals, and why standard correlation may sometimes give misleading results. To solve this issue, normalized correlation is introduced as a more reliable method.

Finally, MATLAB is used to analyze signals composed of multiple sinusoidal components, in order to compare standard and normalized correlation and determine which method better reflects the strength of a signal's presence.

2. Cross-Correlation and Autocorrelation

Cross-correlation and autocorrelation are used in DSP to measure how similar signals are.

Cross-correlation measures the similarity between two different signals. It works by shifting one signal in time and checking how well it matches the other signal at each shift.

Mathematically, it is defined as:

$$R_{x,y}(k) = \sum_{-\infty}^{\infty} x[n]y[n+k]$$

Where k represents the shift (or delay).

If the value of correlation is high at a certain k, this means the two signals are very similar at that shift. This helps in detecting if a signal exists inside another signal and finding its delay.

Autocorrelation is a special case of cross-correlation where a signal is compared with itself:

$$R_{x,x}(k) = \sum_{-\infty}^{\infty} x[n]x[n+k]$$

It is mainly used to detect repeating patterns, find periodic signals and measure signal energy.

The maximum value of autocorrelation always occurs at k = 0, because the signal perfectly matches itself.

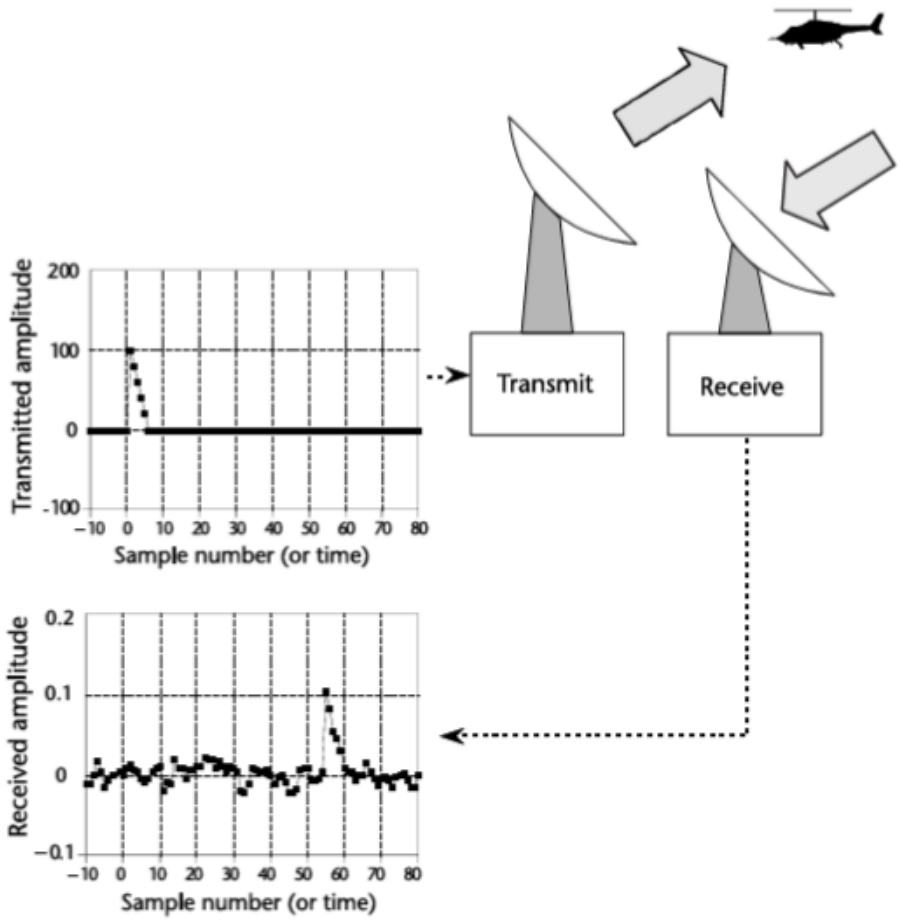


Figure Key elements of a radar system. Like other echo location systems, radar transmits a short pulse of energy that is reflected by objects being examined. This makes the received waveform a shifted version of the transmitted waveform, plus random noise. Detection of a known waveform in a noisy signal is the fundamental problem in echo location. The answer to this problem is *correlation*.

The figure shows a simple example of how correlation is used in a radar system.

The top plot represents the transmitted signal (a short pulse).

This signal travels, hits an object (like a helicopter), and is reflected back.

The bottom plot shows the received signal, which is a delayed and noisy version of the transmitted signal.

Because of noise, it is difficult to directly see where the signal appears. This is where cross-correlation is useful.

By correlating the received signal with the transmitted signal:

- we can detect the presence of the signal

- we can find the delay (time shift)
- we can identify the position of the object

3. Task 1: Correlation vs. Convolution

Correlation and convolution are closely related operations in Digital Signal Processing, and both involve combining two signals. However, they are used for different purposes and have slightly different mathematical forms.

3.1. Mathematical relation

The mathematical expressions of convolution and correlation are very similar.

- Convolution:

$$(x * y)(k) = \sum_{n=-\infty}^{\infty} x[n]y[k-n]$$

- Correlation:

$$R_{x,y}(k) = \sum_{-\infty}^{\infty} x[n]y[n+k]$$

Cross-correlation is equivalent to the convolution of one signal with the time-reversed (and complex-conjugated, for complex signals) version of the other. Mathematically:

$$R_{x,y}(k) = x[n] * y[-n]$$

This means that cross-correlation is equivalent to convolution with one signal reversed in time.

3.2. Key difference

Although they look similar, there are important differences:

- Signal flipping:
  In convolution, one signal is reversed before shifting. In correlation, no flipping is performed.

- Purpose:
  Convolution is mainly used to determine the output of a system when an input signal is applied. Correlation is used to measure the similarity between signals.

- Interpretation:
  Convolution describes how a system modifies a signal.
  Correlation checks how well two signals match.

- Commutativity:
  Convolution is commutative: $x * y = y * x$
  Correlation is not strictly commutative: $R_{x,y}(k) = R_{y,x}(-k)$
  This means switching the signals changes the result unless the shift is also reversed.

- Result at k = 0:
  The value of the output at k = 0 has a different interpretation in correlation and convolution.
  * In correlation:

  $$R_{x,y}(0) = \sum x[n]y[n]$$

  This represents the direct similarity between the two signals without any shift.
  It compares each sample of x[n] with the corresponding sample of y[n].
  If the signals are similar → the value is large and positive.
  If they are different → the value is small or negative.

  For autocorrelation:

$$R_{x,x}(0) = \sum x^2[n]$$

This is the energy of the signal, and it is always the maximum value.

* In convolution:

$$(x * y)(0) = \sum x[n]\, y[-n]$$

Here, one of the signals is reversed in time before multiplication. So at k = 0, convolution does not compare matching samples directly, but instead compares x[n] with y[−n] (a flipped version of the signal).

3.3. Typical applications

Convolution is mainly used to analyze how signals are modified by systems:

- LTI Systems (Filtering):
  Convolution is used to calculate the output of a system when an input signal is applied. This is the foundation of digital filters.
- Signal Filtering:
  It is used to remove noise or enhance certain parts of a signal.
- Image Processing:
  Convolution is applied using kernels (filters) for operations such as blurring, sharpening, and edge detection.
- Polynomial Multiplication:
  The coefficients of the product of two polynomials are obtained using convolution.
- System Response Analysis:
  It helps in understanding how a system reacts to different inputs.

Correlation is mainly used to measure similarity and detect signals:

- Signal Detection:
  Used to determine if a known signal exists within a noisy signal.

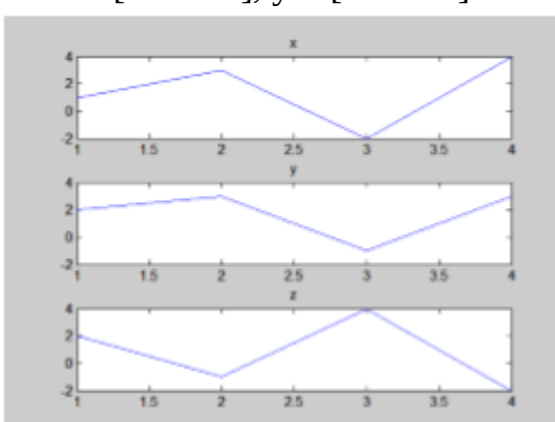
- Pattern Recognition:
  Helps in finding a known pattern or template inside a larger signal.
- Radar and Sonar Systems:
  Used to detect objects and measure distance by comparing transmitted and received signals.
- Synchronization:
  Used in communication systems to detect the start of a signal or data frame.
- Feature Matching:
  Used to compare signals or features and determine how similar they are.

4. Similarity Detection Problem

Correlation is used to measure how similar two signals are. For time-limited signals, the similarity at zero shift (k = 0) is given by:

$$R_{x,y}(0) = \sum_{n=0}^{N-1} x[n]y[n]$$

❖ Example:
Let x = [1 3 -2 4], y = [2 3 -1 3] and z = [2 -1 4 -2]



Rx,y(0) = 1*2 + 3*3 + -2*-1 + 4*3
Rx,y = 25

$R_{x,z}(0) = 1*2 + 3*-1 + -2*4 + 4*-2$
$R_{x,z} = -17$

$R_{x,y}$ is positive and large → strong similarity
$R_{x,z}$ is negative → signals are different (opposite behavior)
So, signal y is more similar to x than z, which is correct.

- Question:
  If we modify z to w = [100 -1 4 -2]:
  1. Repeat correlation with x
  2. Explain why correlation gives misleading results

  $R_{x,w}(0) = 1*100 + 3*-1 + -2*4 + 4*-2$
  $R_{x,w} = 81$

  $R_{x,w} > R_{x,y}$
  This incorrectly suggests that w is more similar to x than y, which is not true.
  This happens because standard correlation depends on amplitude.
  The large value (100) in w dominates the result, even if the overall shape is different, the correlation becomes large.
  Therefore, standard correlation can give misleading results when signal amplitudes are very different (a signal with a very high energy (large amplitudes) can yield a high correlation value even if its "shape" or "pattern" does not match the reference signal).
  To avoid this problem, normalized correlation is used, which removes the effect of amplitude.

5. Normalized Correlation

This method removes the effect of amplitude and focuses only on the shape similarity.

Formula:

$$\sigma_{x,y} = \frac{\sum_{n=0}^{N-1} x[n]y[n]}{\sqrt{\sum_{n=0}^{N-1} x^2[n]}\sqrt{\sum_{n=0}^{N-1} y^2[n]}}$$

- Compute normalized correlation between:
  1. x & y
  2. x & z
  3. x & w

σx, = (1*2 + 3*3 + -2*-1 + 4*3) / ((√(1^2 + 3^2 + (-2)^2 + 4^2))(√(2^2 + 3^2 + (-1)^2 + 3^2)))

σx, = 25 / (5.47723 * 4.796)

σx, = 0.9517 → very strong similarity

σx,z = (1*2 + 3*-1 + -2*4 + 4*-2) / ((√(1^2 + 3^2 + (-2)^2 + 4^2))(√(2^2 + (-1)^2 + 4^2 + (-2)^2)))

σx,z = -17 / (5.47723 * 5)

σx,z = -0.620752 → dissimilar signals

σx,w = (1*100 + 3*-1 + -2*4 + 4*-2) / ((√(1^2 + 3^2 + (-2)^2 + 4^2))(√(100^2 + (-1)^2 + 4^2 + (-2)^2)))

σx,w = 81 / (5.47723 * 100.105)

σx,w = 0.147729855 → weak similarity

Signal y is correctly identified as most similar to x. While signal w, despite its large values, shows low similarity and signal z is clearly different.

- Compare results and explain:

1. Why normalization improves detection?

Normalization improves detection because it removes the effect of the signal's amplitude (or energy), so large values no longer dominate the result. Instead of being affected by how "strong" or "loud" a signal is, the comparison focuses only on the shape or pattern. It also scales the result to lie between −1 and 1, where 1 means perfect similarity, 0 means no similarity, and −1 means opposite behavior. This makes the result easier to understand and more reliable, since it reflects true similarity based on the signal's shape rather than its magnitude.

2. When normalized correlation should be used?
Normalized correlation should be used whenever the goal is to compare signals based on their shape or pattern, not their absolute power. This is especially important when signals may have different amplitudes, such as in noisy environments or when signal strength changes (for example, due to distance in radar or volume in audio). It is commonly used in applications like image template matching and speech recognition, where accurate similarity detection is needed even if the signal levels are not consistent.

6. Task 2: MATLAB Analysis

Let Si= cos(2*π*fi*t)

s1: 1 Hz s2: 4 Hz s3: 10 Hz

The following signals contain the three sinusoids above

X = 2S1+ 4S2 + S3

Y = S1+ S2

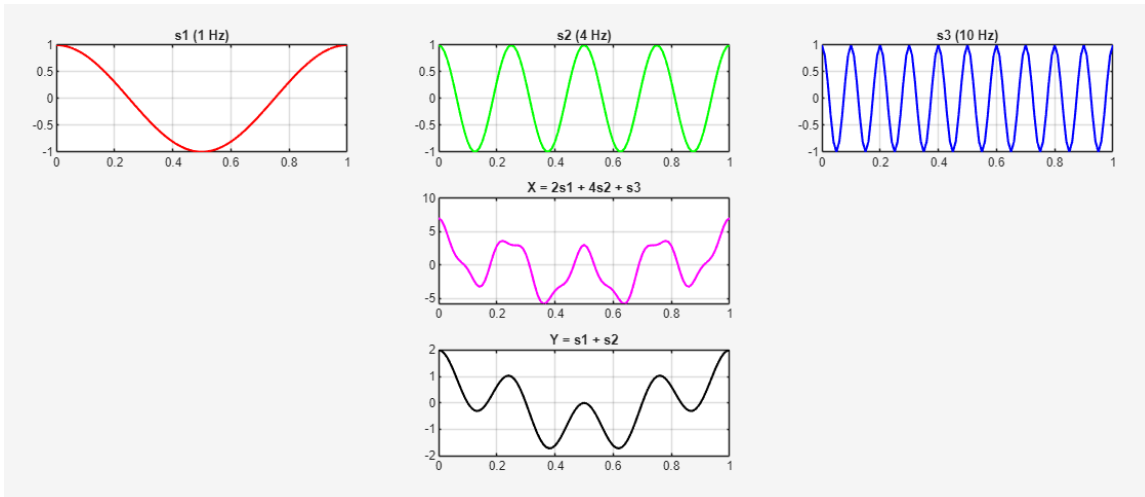
a) Plot the generated signals for 1 sec. let the sampling interval 0.01 sec
   Code:

```
t = 0:0.01:1 ;  % Create time vector from 0 to 1
second with step (sampling interval) = 0.01

% Signals:
```

```
s1 = cos(2*pi*1*t) ;  % Generate s1: cosine signal
with frequency 1 Hz

s2 = cos(2*pi*4*t) ;  % Generate s2: cosine signal
with frequency 4 Hz

s3 = cos(2*pi*10*t) ;  % Generate s3: cosine
signal with frequency 10 Hz

X = 2*s1 + 4*s2 + s3 ;  % Create signal X as a
combination of signals: 2*s1 (scaled), 4*s2
(stronger scaling), and s3

Y = s1 + s2 ;  % Create signal Y as a combination
of s1 and s2

figure('Position', [100 , 100 , 900 , 700]) ;  %
Open a new figure window and set its size and
position on the screen

% --- Row 1 ---
subplot(3 , 3 , 1) ;  % Divide figure into 3x3
grid, select position 1 (top-left)

plot(t , s1 , 'r' , 'LineWidth' , 1.5) ;  % Plot
s1 versus time in red color with thicker line

title('s1 (1 Hz)') ;  % Add title to the s1 plot

grid on ;  % Turn on grid for better visualization

subplot(3 , 3 , 2) ;  % Select position 2

plot(t , s2 , 'g' , 'LineWidth' , 1.5) ;  % Plot
s2 in green with thicker line

title('s2 (4 Hz)') ;  % Add title to the s2 plot

grid on ;  % Enable grid
```

```
subplot(3 , 3 , 3) ;  % Select position 3 (top-
right)

plot(t , s3 , 'b' , 'LineWidth' , 1.5) ;  % Plot
s3 in blue with thicker line

title('s3 (10 Hz)') ;  % Add title to the s3 plot

grid on ;  % Enable grid

% --- Row 2 (X in the middle) ---
subplot(3 , 3 , 5) ;  % Select position 5 (center
of the grid → middle of second row)

plot(t , X , 'm' , 'LineWidth' , 1.5) ;  % Plot X
in magenta color with thicker line

title('X = 2s1 + 4s2 + s3') ;  % Add descriptive
title showing how X is formed

grid on ;  % Enable grid

% --- Row 3 (Y in the middle) ---
subplot(3 , 3 , 8) ;  % Select position 8 (center
of third row)

plot(t , Y , 'k' , 'LineWidth' , 1.5) ;  % Plot Y
in black color with thicker line

title('Y = s1 + s2') ;  % Add descriptive title
for Y

grid on ;  % Enable grid
```

Plots:



b) Compute:

1. Standard correlation between X and s1
2. Normalized correlation between X and s1

Code:

```
t = 0:0.01:1 ;  % Create time vector from 0 to 1
second with step (sampling interval) = 0.01

% Signals:
s1 = cos(2*pi*1*t) ;  % Generate s1: cosine signal
with frequency 1 Hz

s2 = cos(2*pi*4*t) ;  % Generate s2: cosine signal
with frequency 4 Hz

s3 = cos(2*pi*10*t) ;  % Generate s3: cosine
signal with frequency 10 Hz

X = 2*s1 + 4*s2 + s3 ;  % Create signal X as a
combination of signals: 2*s1 (scaled), 4*s2
(stronger scaling), and s3

Y = s1 + s2 ;  % Create signal Y as a combination
of s1 and s2

% --- Standard Correlation between X and s1 (at k
= 0) ---
```

```
R_X_s1 = sum(X .* s1) ;  % Multiply element-by-
element and sum

% --- Normalized Correlation between X and s1 ---
R_X_s1_norm = R_X_s1 / sqrt(sum(X.^2) *
sum(s1.^2)) ;  % Divide by signal energies(removes
amplitude effect)

% --- Display results ---
disp('Standard Correlation R_X_s1 =') ;
disp(R_X_s1) ;
disp('Normalized Correlation R_X_s1_norm =') ;
disp(R_X_s1_norm) ;
```

Output:

```
Command Window
Standard Correlation R_X_s1 =
  107.0000

Normalized Correlation R_X_s1_norm =
    0.4520
```

c) Repeat for Y and X and s1

Code:

```
t = 0:0.01:1 ;  % Create time vector from 0 to 1
second with step (sampling interval) = 0.01

% Signals:
s1 = cos(2*pi*1*t) ;  % Generate s1: cosine signal
with frequency 1 Hz

s2 = cos(2*pi*4*t) ;  % Generate s2: cosine signal
with frequency 4 Hz

s3 = cos(2*pi*10*t) ;  % Generate s3: cosine
signal with frequency 10 Hz
```

```
X = 2*s1 + 4*s2 + s3 ;  % Create signal X as a
combination of signals: 2*s1 (scaled), 4*s2
(stronger scaling), and s3

Y = s1 + s2 ;  % Create signal Y as a combination
of s1 and s2

% --- Standard Correlation between Y and s1 ---
R_Y_s1 = sum(Y .* s1) ;  % Multiply element-by-
element and sum

% --- Normalized Correlation between Y and s1 ---
R_Y_s1_norm = R_Y_s1 / sqrt(sum(Y.^2) *
sum(s1.^2)) ;  % Divide by signal energies(removes
amplitude effect)

% --- Display results ---
disp('Standard Correlation R_Y_s1 =') ;
disp(R_Y_s1) ;
disp('Normalized Correlation R_Y_s1_norm =') ;
disp(R_Y_s1_norm) ;
```

Output:

```
Command Window
Standard Correlation R_Y_s1 =
   52.0000

Normalized Correlation R_Y_s1_norm =
    0.7140
```

d) Which method better reflects the strength of presence of s1?
Normalized correlation better reflects the strength of the presence of s1 in the signals. This is because it removes the effect of amplitude and focuses only on the similarity in shape between the signals. As a result, it provides a fair comparison and accurately shows how much s1 is actually present in each signal, regardless of scaling.

e) Why does standard correlation depend on amplitude?

Standard correlation depends on amplitude because it is calculated by multiplying corresponding values of the signals and summing them. If a signal has large amplitudes, the products become larger, which increases the overall result. This means that even if two signals are not very similar in shape, a signal with higher amplitude can still produce a large correlation value, making the result misleading.

f) Which method would you use in:

1. Signal detection?

Standard correlation can be used when the amplitude (or energy) of the signal is important, such as detecting whether a strong signal is present.

2. Pattern recognition?

Normalized correlation should be used because it focuses on the shape or pattern of the signal and ignores differences in amplitude, making it more accurate for comparing similarity.

- ✓ Normalized correlation is generally preferred when accurate similarity measurement is required.

- Analysis:

In standard correlation:

$R_{Y,s1} = 52$

$R_{X,s1} = 107$

This suggests that s1 is more present in X than in Y, because $107 > 52$

However, this conclusion is misleading.

Because signal X contains scaled components: $X = 2S1 + 4S2 + S3$

The larger amplitudes increase the overall values of X, which leads to a larger correlation result, even if the actual similarity to s1 is not stronger.

So, standard correlation is affected by amplitude, not just similarity.

In normalized correlation:

$R_{Y,s1} = 0.714$

$R_{X,s1} = 0.452$

Y has a higher normalized correlation with s1.

This means:

Y is more similar in shape to s1 and s1 is actually more strongly present in Y than in X.
Which is correct because normalization removes the effect of amplitude and focuses only on shape similarity, giving a more accurate result.

7. Conclusion

This assignment explored the fundamental concepts of correlation and convolution in Digital Signal Processing (DSP), focusing on their differences and practical uses. Convolution, which involves time-reversing one signal before combining it with another, is mainly used in analyzing Linear Time-Invariant (LTI) systems and modeling operations such as filtering. In contrast, correlation does not involve time reversal and is used to measure the similarity between signals, making it useful for applications like signal detection, pattern recognition, and synchronization.

An important concept highlighted in this assignment is the limitation of standard correlation when signal amplitudes vary. It was shown that signals with larger amplitudes can produce higher correlation values even if their shapes are not very similar, which can lead to incorrect conclusions. To solve this problem, normalized correlation was introduced. By scaling the result between -1 and 1, normalized correlation removes the effect of amplitude and provides a more accurate measure of similarity based only on the signal shape.

The MATLAB analysis in Task 2 confirmed these theoretical concepts. The results showed that standard correlation is strongly affected by amplitude, while normalized correlation gives a more reliable indication of how strongly a signal is present within another. This demonstrates the importance of choosing the appropriate method depending on the goal of the analysis.

In conclusion, standard correlation is useful when signal strength or energy is important, while normalized correlation is more suitable for comparing signal patterns and detecting similarity accurately.

Understanding the difference between these methods is essential for effective signal analysis in real-world applications.